

Module 20: OOP Principles & Object Modelling

Introduction to C++ Programming — Part 3: Object-Oriented Programming with C++

Module Overview

This module opens Part 3 by changing the question students ask. Parts 1 and 2 taught the mechanics of classes, inheritance and polymorphism; Part 3 is about design - deciding which classes a program needs, what each is responsible for, and how they relate. No new syntax is introduced. Instead, students compare procedural and object-oriented versions of the same program, name the four pillars of OOP, and practise turning a plain-language problem statement into classes with noun/verb analysis, CRC cards and text UML. The module also introduces the Part 3 compile command with `-std=c++17 -Wall -Wextra` and the habit of treating warnings as bugs.

Suggested Duration: 75–90 minutes

Learning Objectives

By the end of this module, students will be able to:

- Compile with an explicit language standard and warning flags, and explain why a clean build has zero warnings.
- Compare a procedural and an object-oriented solution to the same problem and explain the trade-offs.
- Name and illustrate the four pillars of OOP: encapsulation, abstraction, inheritance and polymorphism.
- Use noun/verb analysis to identify candidate classes, attributes and methods from a problem statement.
- Write a CRC card for a class and read and draw a simple text UML class diagram.

Prerequisites

- Part 1 Module 8 (Classes and Objects) and Part 2 Modules 11-13 (Inheritance, Polymorphism, Operator Overloading).
- Students should have completed (or at least seen) Capstone Project 2 in Module 19.

Materials & Instructor Prep

- 20-oop-principles/examples/01_procedural_version.cpp and 02_object_version.cpp open side by side.
- A whiteboard or flipchart for the live noun/verb analysis - this is a paper-first module.
- Index cards (or sticky notes) for CRC cards: one per class, enough for groups of three.

Key Vocabulary

Term	Definition
Encapsulation	Bundling data with the methods that act on it, and keeping the data private so outside code must use those methods.
Abstraction	Showing what an object does while hiding how it does it, so users of a class only need to know its interface.
Inheritance	A derived class reusing and extending a base class to model an "is-a" relationship.
Polymorphism	One interface, many behaviours: code written against a base class works with every derived class through virtual dispatch.
Noun/verb analysis	Underlining the nouns (candidate classes/attributes) and verbs (candidate methods) in a problem statement.
CRC card	Class-Responsibilities-Collaborators: a small card listing what a class knows and does, and which classes it works with.
UML class diagram	A standard drawing of classes, their members (+ public, - private, # protected) and the relationships between them.

Lesson Outline & Instructor Talking Points

1. Welcome to Part 3 and the new compile command (10 min)

- Frame Part 3 explicitly: 'You already know how to write a class. Now you will learn to design with them.' Connect back to Capstone 2's biggest class and ask how many responsibilities it had.
- Introduce `g++ -std=c++17 -Wall -Wextra`. Compile a small file with an unused variable and an uninitialised one, and show the warnings appear. State the rule: a clean build has zero warnings.

Instructor note: Some students will see warnings as 'not real errors'. Settle this now - every warning in Part 3 work is treated as a bug to fix.

2. Procedural vs object-oriented: the library example (20 min)

- Run `examples/01_procedural_version.cpp`. Point at the parallel vectors and ask: what stops a programmer from adding a title without a matching borrowed flag? (Nothing.) Show the line that bypasses `borrowBook()`.
- Run `examples/02_object_version.cpp`. Show that the same misuse no longer compiles, and that the `Book` itself refuses a second borrow.
- Ask the class which version they would rather change in six months when the library adds due dates. Make clear neither style is 'wrong' - OOP earns its place as programs grow.

Instructor note: Resist turning this into 'procedural bad, OOP good'. The honest message is about cost of change, which is the theme of all of Part 3.

3. The four pillars (15 min)

- Walk through examples/03_four_pillars.cpp, pausing on each labelled comment. For each pillar, ask students to name the module where they first met it (8, 11, 12).
- Spend extra time separating encapsulation (protecting data) from abstraction (simplifying what users must think about) - students often treat them as synonyms.

4. Live noun/verb analysis: the school fees system (20 min)

- Write the school fees problem statement from the README on the board. Have students call out nouns and verbs while you underline them in two colours.
- Build the decision table together: class, attribute, or neither. Make the 'balance' judgement call explicitly - storing it separately recreates the parallel-vector problem.
- Run examples/04_from_nouns_to_classes.cpp to show the table turned into code.

Instructor note: *This is the load-bearing activity of the module. The goal is the reasoning, not one 'correct' list - accept defensible alternatives and ask students to justify them.*

5. CRC cards and text UML (10 min)

- In groups of three, students write a CRC card for FeeAccount and one other class, then draw both as text UML boxes with + and - markers.
- Show the inheritance arrow and tell students the other relationship arrows arrive in Module 27.

Common Student Misconceptions

Misconception	How to Address It
"Every noun should become a class."	Most nouns become attributes or nothing at all. A class needs data AND behaviour or rules; a single value like a name or an amount usually doesn't.
"Encapsulation and abstraction are the same thing."	Encapsulation protects data behind methods; abstraction simplifies what a user needs to know. A class can be well encapsulated but still expose a confusing interface.
"Warnings don't matter if it compiles."	Warnings flag code that is legal but probably wrong - uninitialised variables, unused parameters, sign mismatches. In Part 3 a clean build means zero warnings.
"OOP means everything must be a class."	Small helper calculations can stay as free functions. OOP is a tool for managing rules and change, not a requirement to wrap every line in a class.

Exercises & Module Project

Exercise: 20-oop-principles/exercises/exercise1.cpp - noun/verb analysis of an intercity bus ticketing statement, then Ticket and Bus classes that refuse duplicate and out-of-range seats. Project (three tracks, continued through all of Part 3): Generic - School Club Registry; EE - Home Energy Monitor; Biomedical - Clinic Appointment System. Each track writes DESIGN.md (noun/verb table, CRC cards, text UML) before coding.

Facilitation guidance: Check DESIGN.md before looking at any code - the project is mostly design. Look for classes that hold both data and rules, derived values calculated rather than stored, and every class box showing + and - markers. Remind students that the track they pick now continues to Capstone Project 3, so it is worth choosing carefully.

Discussion & Comprehension Check Questions

1. Why might storing a 'balance' field alongside a list of payments cause bugs?

Guidance: The two can disagree: any code path that adds a payment but forgets to update the balance leaves the object inconsistent. Calculating the balance from the payments makes that bug impossible.

2. What is the difference between encapsulation and abstraction?

Guidance: Encapsulation keeps data private and changes it only through methods that enforce rules; abstraction presents a simple interface that hides how the work is done.

3. In the school fees statement, why is the bursar NOT a class?

Guidance: The bursar is the user of the program, not something the program needs to model or store data about.

Assessment Ideas

- Give a new short problem statement (e.g. a canteen ordering system) and ask students to produce a noun/verb table and two CRC cards in 15 minutes.
- Show a text UML diagram and ask students to write the matching class skeletons in C++ with correct access specifiers.

Answer Key Notes

See 20-oop-principles/exercises/solutions/exercise1_solution.cpp, which includes a sample noun/verb analysis in its opening comment.

Wrap-Up & Bridge to Next Module

Bridge forward: "You can now decide which classes a program needs. Next module goes back inside a single class and makes it hard to misuse: initializer lists, const correctness, invariants and static members."

Instructor Tips

- If time is short, protect the live noun/verb analysis (step 4) - it is the skill every later Part 3 project depends on.
- Encourage paper first. Students who jump straight to code tend to produce one giant class.